盐城超算上机使用手册 V0.3.1

一 使用概述

1、超算组成

- 登录服务器。**专门给用户远程登录使用的服务器**。用户凭超算账号及密码登入服务器，然后在服务器上可进行**文件上传下载、文件编辑、程序编译、软件安装、计算任务提交**等操作，但**不能直接运行计算任务**，否则会导致机器卡顿，影响其他用户登录及使用；
- 计算服务器。**专门用来运行计算任务的服务器**，数量多、性能强。
- 存储系统。**并行存储系统**，使用上可看成网盘，在登录服务器和计算服务器都有挂载，挂载的目录名为：`online1`。**数据要放在 `online1` 目录下才能保证良好的读写性能**，以及登录服务器和计算服务器都可以访问操作；
- 调度系统。所有计算服务器由调度系统分配管理。用户首先向调度系统申请计算资源，然后再由调度系统将计算任务投放到分配的计算服务器上运行。

2、使用步骤

1. 用户在外部互联网，通过`VPN`接入超算中心内部局域网，然后使用超算账号进行集群登录
2. 将程序和数据上传到 `online1` 目录
3. 编写计算任务提交脚本（向调度申请计算资源以及程序运行命令）
4. 执行任务提交脚本，将计算任务投放到计算服务器上运行
5. 查看程序运行状态

> 切记所有文件数据都要放在`online1`目录下，才能体验超大存储空间和高速读写性能！

二 系统登录

2.1 登录 VPN

2.1.1 Windows 用户

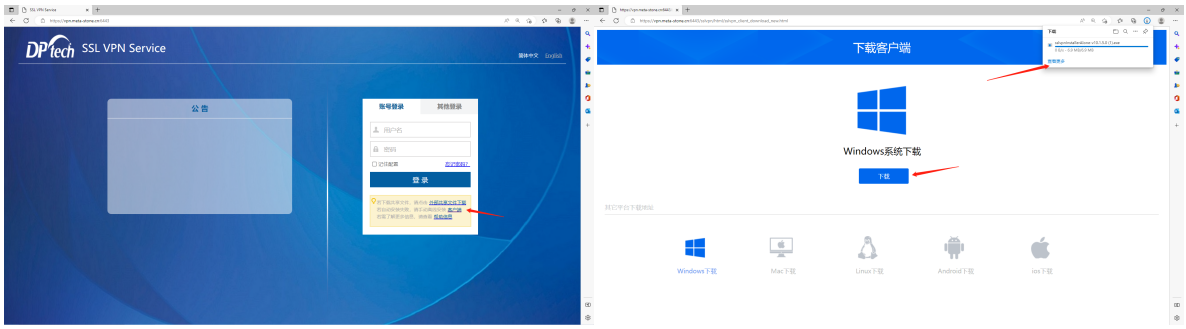
本小节使用 Windows10 进行示例操作

在正式登录超算之前，您需要先登录VPN进入盐城超算平台的内部网络。

步骤一：浏览器访问 VPN 地址

在浏览器地址栏中输入 https://vpn.meta-stone.cn:6443

进入后，点击 “客户端” 字样，选择 Windows 系统下载，点击下载

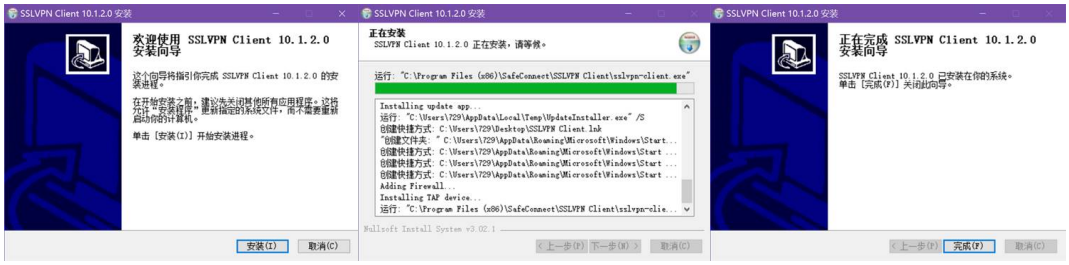


步骤二：Windows 下安装 SSL VPN

双击刚才下载的客户端安装程序

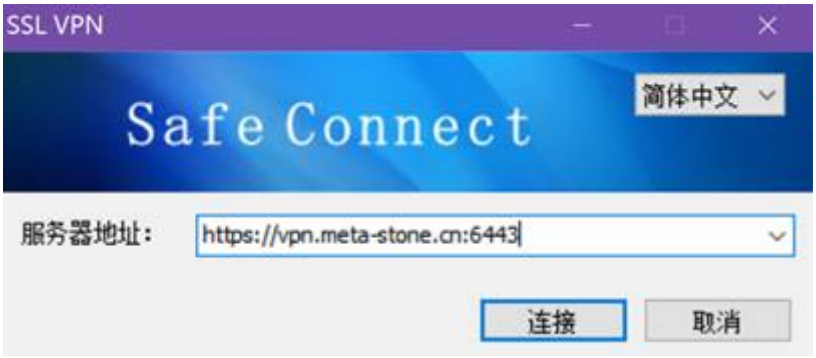
> **Ps:期间若提示网卡适配器安装，一定点击是，否则无法使用**

根据提示安装即可

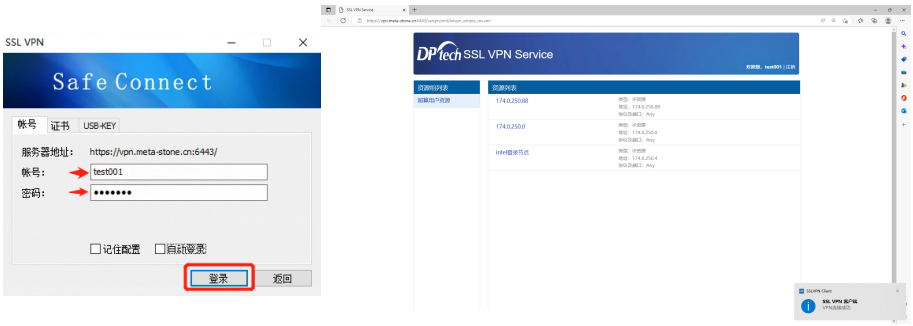


步骤三：登录 SSL VPN

打开刚才安装的客户端程序，服 务器地址处填写 https://vpn.meta-stone.cn:6443/，然后点击连接



然后输入超算中心所分配的用户名密码，点击登录，若登录成功，终端默认浏览器会自动跳出 SSLVPN 资源页面



进行完以上步骤后，即表明顺利登入 VPN 进入盐城超算平台内网

2.1.2 MAC 用户

步骤一：浏览器访问 VPN 地址

在浏览器地址栏中输入 https://vpn.meta-stone.cn:6443 并进入

进入后，点击客户端字样进行下载，下载 MAC 系统客户端



步骤二：macOS 下安装 SSL VPN

点击下载后安装包





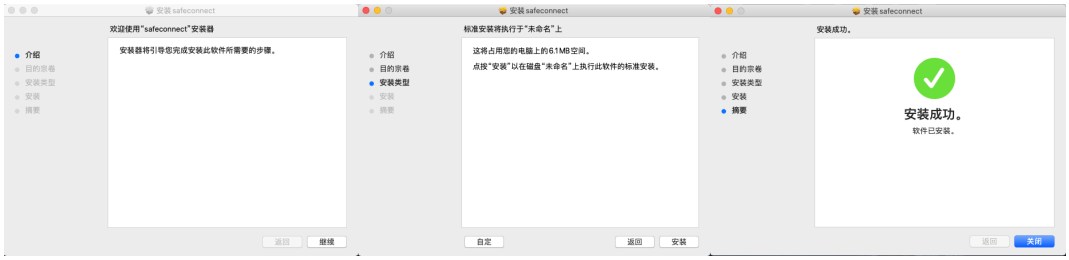
点击问号，在弹出窗口点击 为我打开“通用面板”，然后点击锁按钮



输入帐号密码，点击打开再次确认



点击继续，然后点击安装，安装完毕后点击关闭即可



步骤三：登录 SSL VPN

打开 SSL VPN，在弹出界面输入 https://vpn.meta-stone.cn:6443/ ，然后输入账号密码，点击登录



弹出如下界面即表示登陆成功

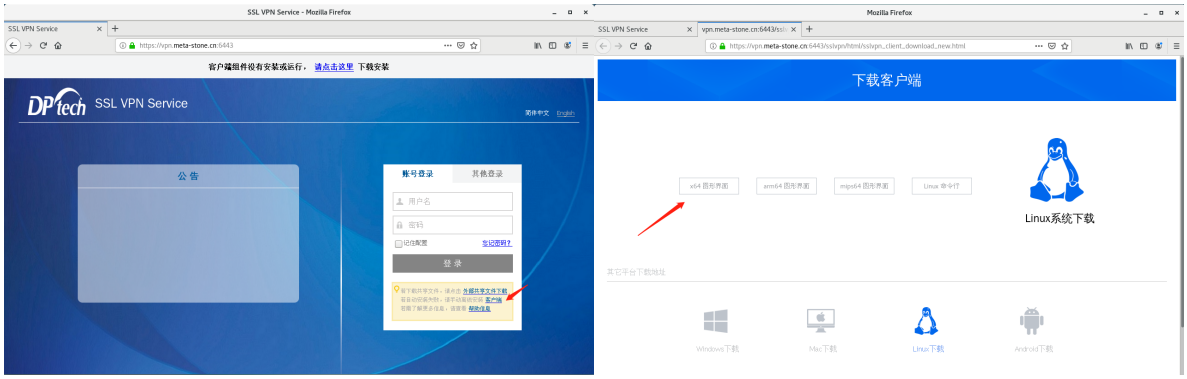


2.1.3 Linux用户

步骤一：浏览器访问 VPN 地址

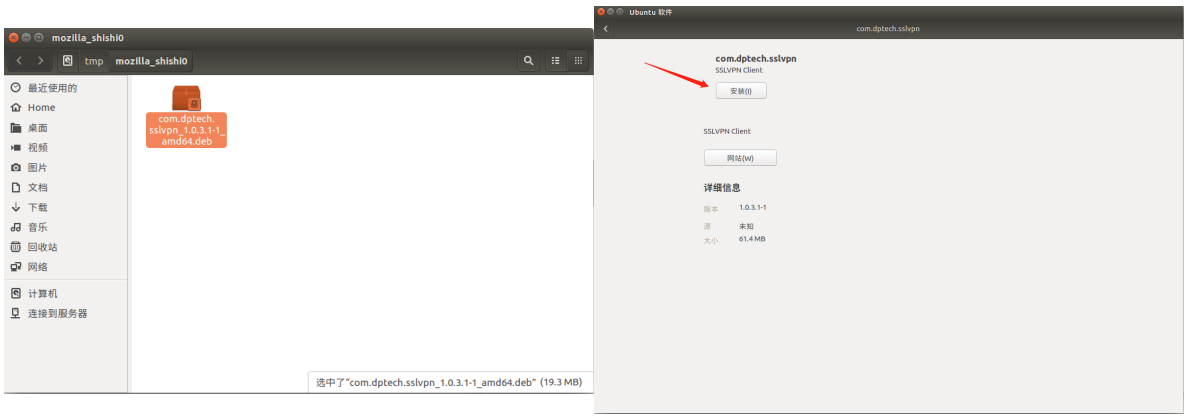
在浏览器地址栏中输入 https://vpn.meta-stone.cn:6443 并进入

进入后，点击客户端字样进行下载



步骤二：Ubuntu 下安装SafeConnect

打开安装包文件夹，右键点击安装



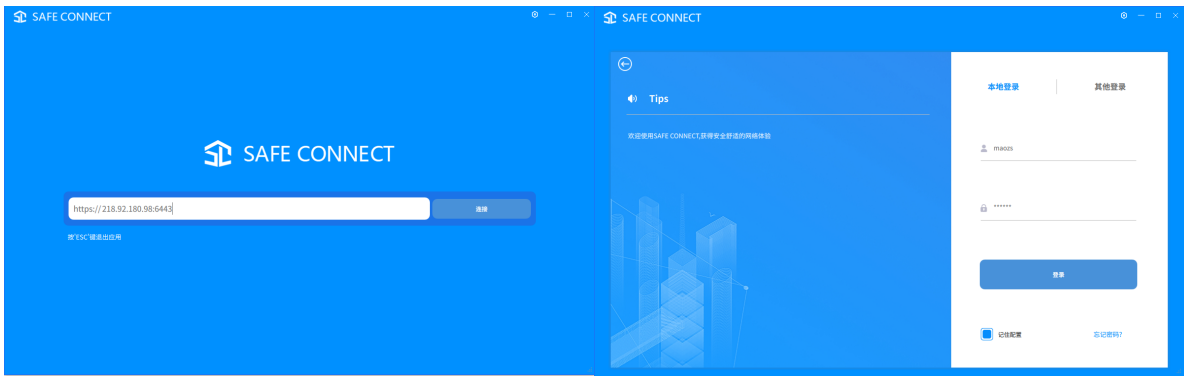
步骤三：登录 SafeConnect

找到 SafeConnect，点击打开

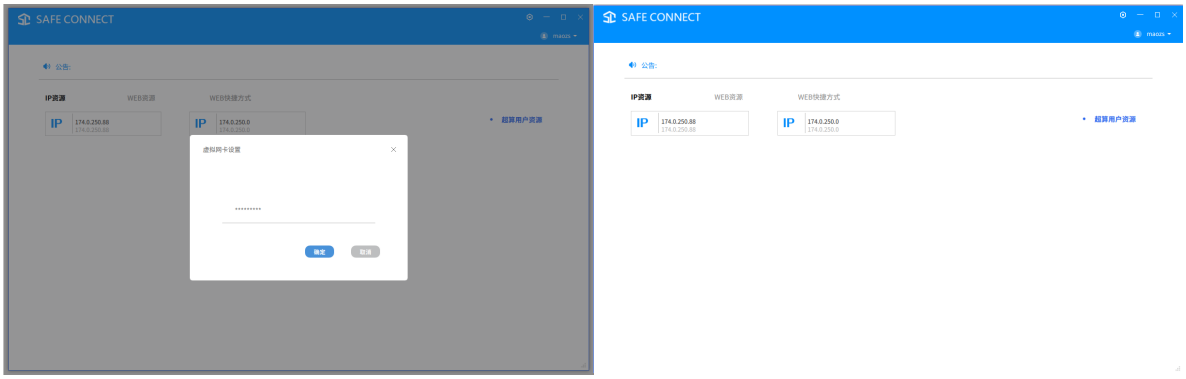


> 地址栏输入以下地址即可
>
> > 223.108.218.34:6443

然后点击连接，然后输入 VPN 账号名和密码，点击登录



输入您当前 Linux 账户的密码，点击确定即可登录成功



命令行终端下载

步骤一：下载 SSLVPNClientLinux 并安装

```
1.下载客户端
wget https://vpn.meta-stone.cn:6443/sslvpn/client/LinuxPlugin.7z

2.下载epel第三方软件库后下载p7zip软件进行解压
yum install -y epel-release
yum install -y p7zip
7za x LinuxPlugin.7z

3.修改文件执行权限
chmod 777 -R ./SSLVPNClientLinux

4.安装客户端
cd ./SSLVPNClientLinux/
./install.sh
```

步骤二：登录客户端

```
1.登录客户端，输入：IP地址、端口号、账户名、密码
sslvpn_client_linux
屏幕输出：
IP Address:221.130.101.33
Port:6443
Username:在此输入您的VPN账户名
Enter Password:在此输入账号密码
```

屏幕输出 VPN network connected! 即为成功登录客户端。

步骤三：退出客户端

```
sslvpn_client_linux -d
```

屏幕输出 VPN network disconnected! 即为成功退出客户端。

2.2 超算登录

如果要远程登录超算，Windows 用户可以使用 Xshell 等终端软件进行连接，此外，还需要安装 Xftp 软件进行文件传输

MAC 和 Linux 用户可以使用操作系统内置的终端和 `scp` 命令进行连接和文件传输

在连接超算之前，请务必保证自己已经登录了 `VPN`

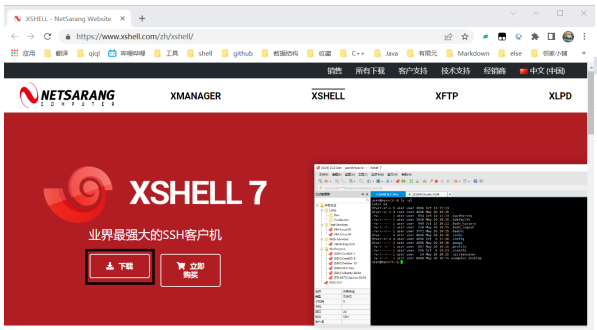
2.2.1 Windows用户

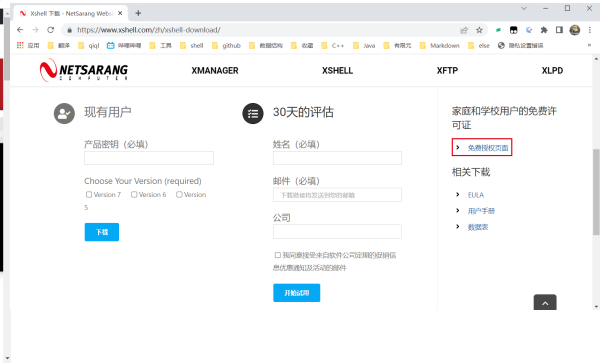
步骤一：安装 Xshell

什么是 Xshell？

Xshell 是一个用来在 Windows 连接远程服务器的终端工具，Xshell 使用方便且免费。

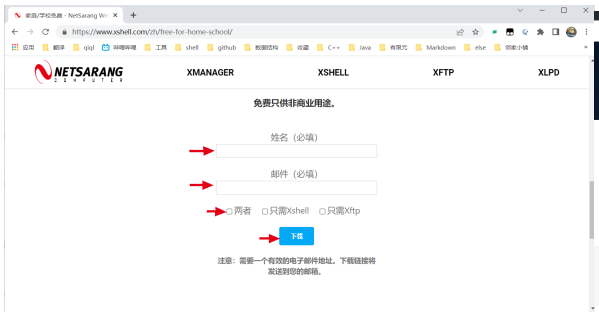
进入XShell的中文官网:https://www.xshell.com/zh/xshell/ 进入如下界面，目前版本为 Xshell7，点击下载，点击免费授权页面

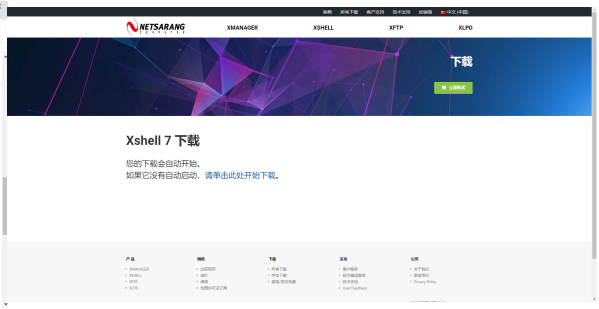




Xshell的优点

进入免费授权页面后，填写姓名和邮件，勾选两者（Xftp 也需要使用），点击下载。注意：需要一个有效的邮件地址，下载链接将发送到邮箱。然后点击邮箱中的下载链接，浏览器将自动下载 Xshell 和 Xftp 的安装包。





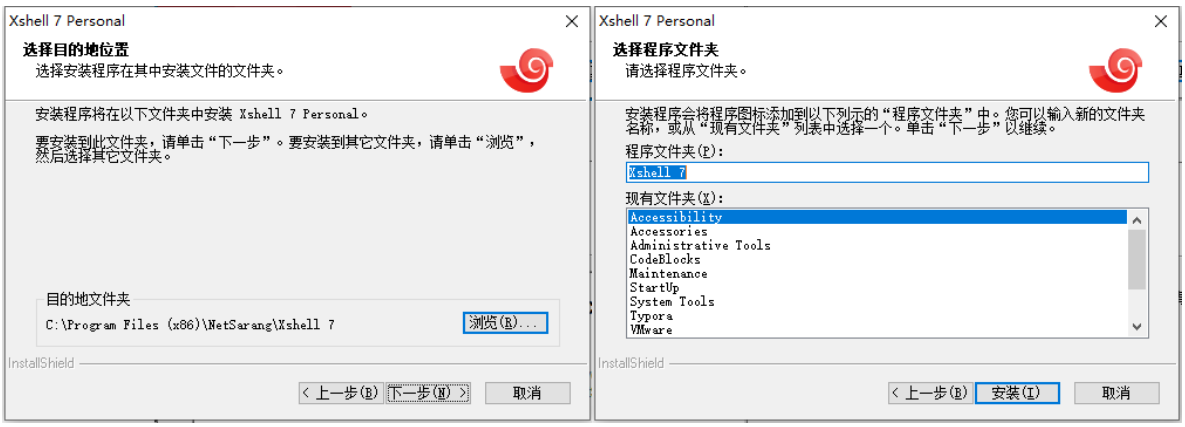
Windows 下安装 Xshell

打开 Xshell7 安装包，点击下一步，勾选我接受，点击下一步





可以选择安装路径或直接点击下一步，点击安装



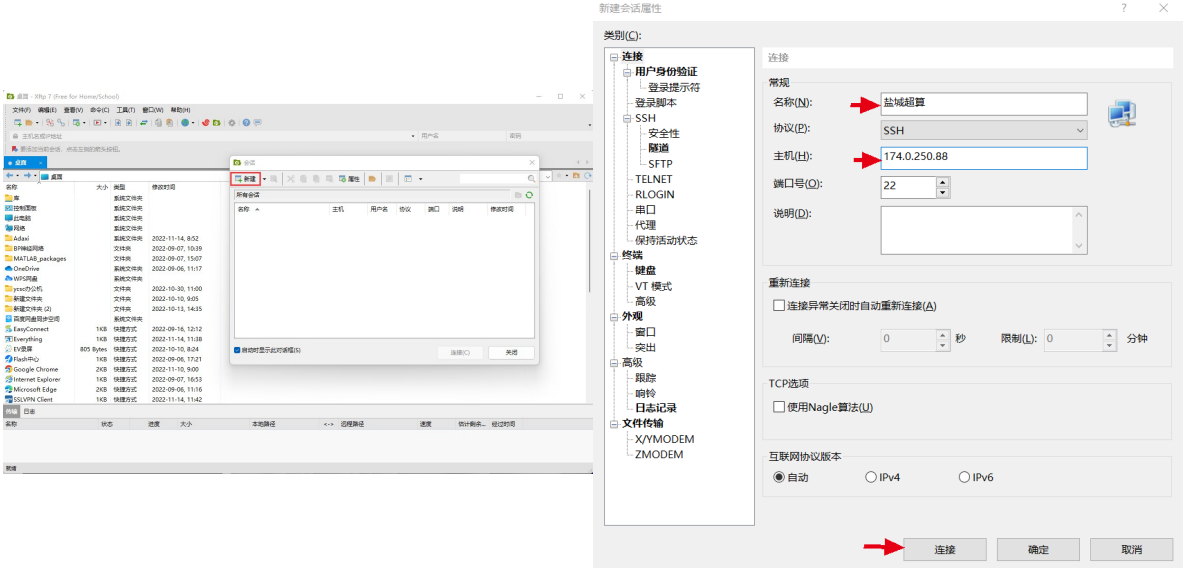
安装完成



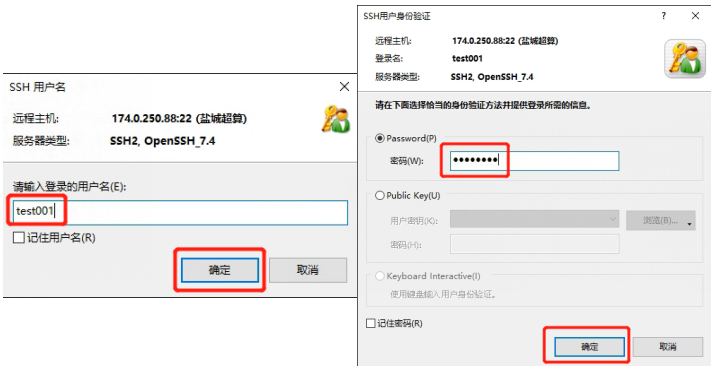
步骤二：Xshell 连接超算

连接超算前您需要先确定已经登录了 VPN

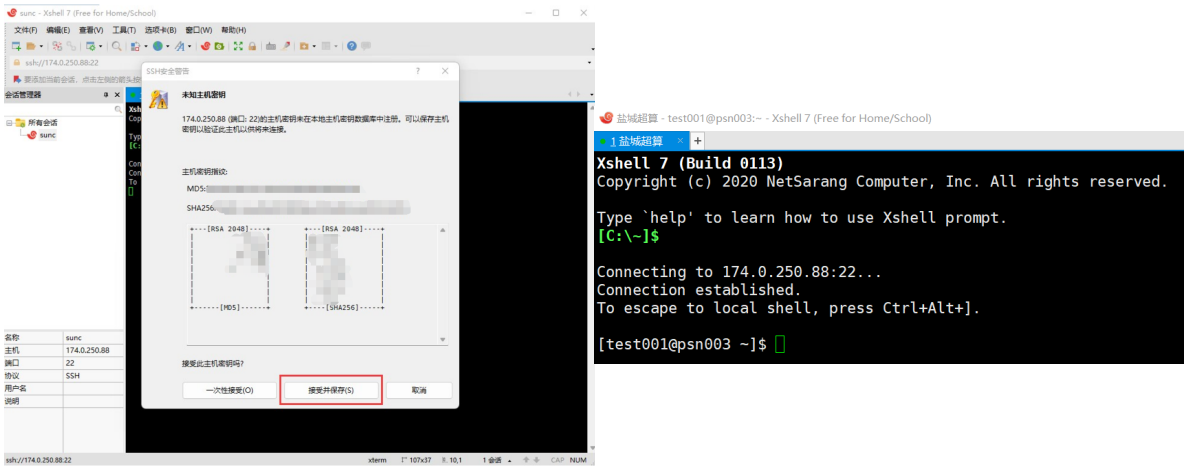
进入软件界面，点击新建，将弹出窗口新建会话，名称可自定义，如盐城超算，主机处填写 174.0.250.88，然后点击连接



在弹出的窗口下输入用户名（此用户名即为所开通的超算账户名），可勾选记住用户名，然后点击确定，然后输入密码，可勾选记住密码，点击确定



在登录后，可能会弹出如图所示的 SSH 安全警告，只会在第一次连接时出现，点击接受并保存，就可以成功登录到超算的登录节点了



连接之后，建议进入 online1 目录再进行操作，home 目录下磁盘大小会进行限额，默认限额为 1GB，online1 目录磁盘免费容量为 1TB，如需扩充，请联系销售

> 注：为方便起见，上述教程采用新建会话方式进行登录，也可以直接在 Windows 的命令行窗口或 Xshell 中直接输入 ssh username@174.0.250.88 登录，username 为所开通的账户名

步骤三：安装 Xftp

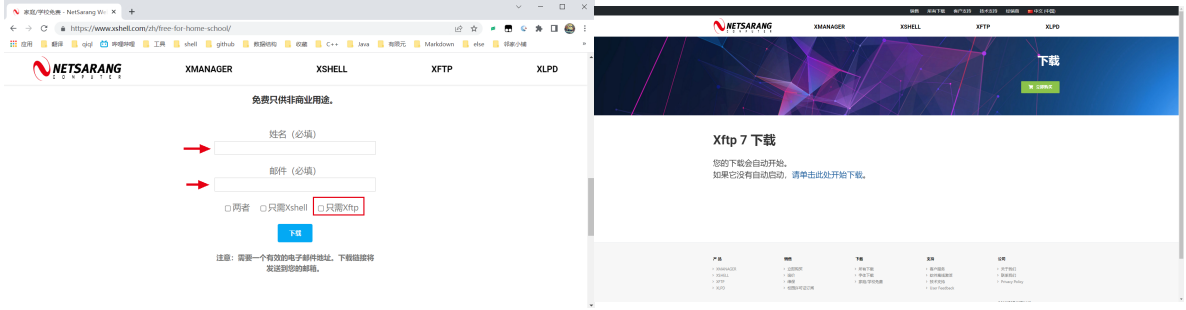
Xftp 的作用：

Xftp 是一个功能强大的 SFTP、FTP 文件传输软件。使用 Xftp 可以方便的在 Windows 主机和远程 Linux 集群之间传文件

浏览器下载 Xftp

如果在刚才下载 Xshell 的页面中已经勾选了两者，那么邮箱中应该还有一个下载Xftp的链接，可以直接点击该链接进行下载，或者在浏览器输入https://www.xshell.com/zh/free-for-home-school/，跳转到如下界面，输入姓名及**有效**邮箱，勾选 Xftp，点击下载

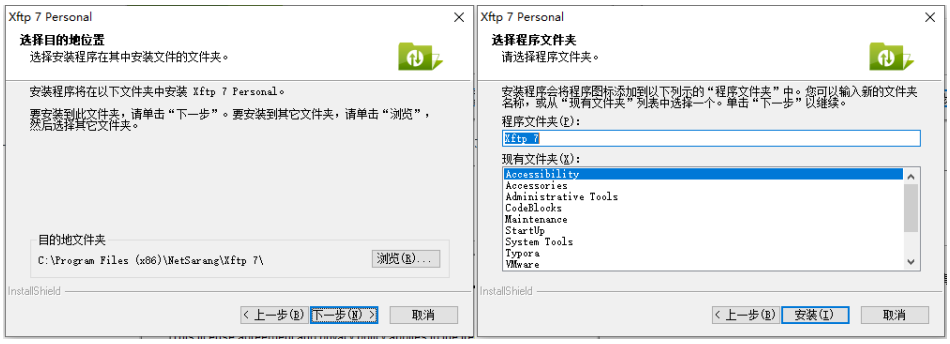
点击邮箱中的下载链接，进入以下界面，浏览器自动下载



打开 Xftp7 安装包，点击下一步，勾选我接受，点击下一步



选择安装路径，也可以不更改安装路径，点击下一步，点击安装

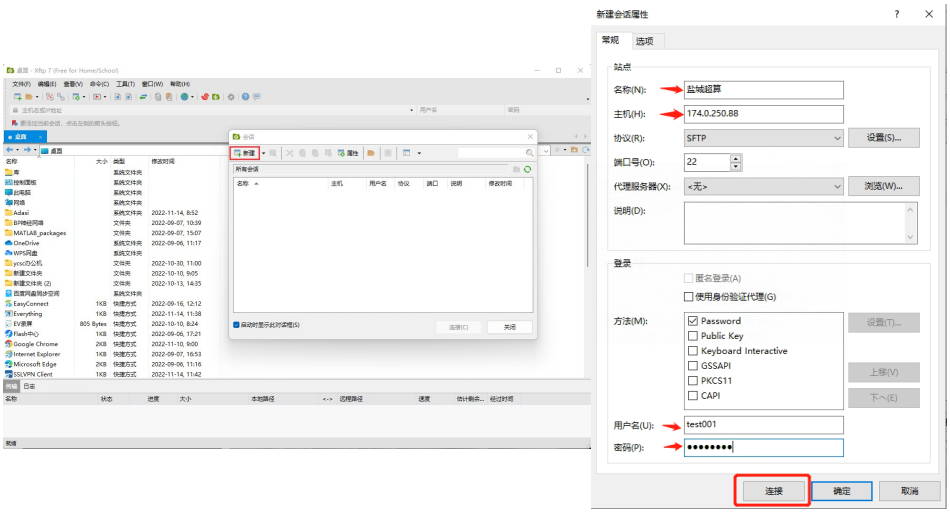


安装完成

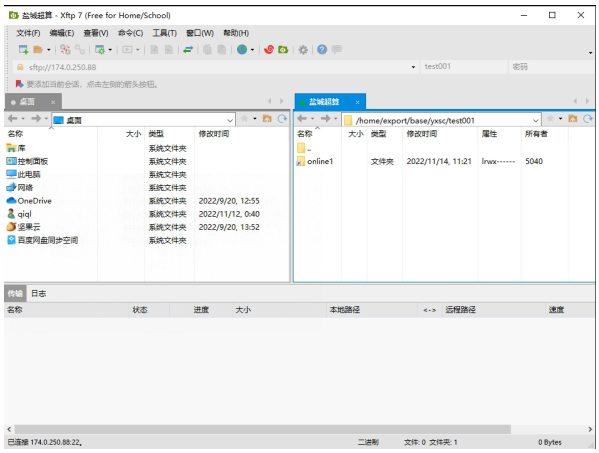


步骤四： Xftp 传文件

进入软件界面，可见 Windows 系统下的文件，点击左上角的新建，弹出如下窗口新建会话，名称可自定义，比如盐城超算，主机处填写174.0.250.88，输入所开通的账户名和密码，然后点击连接



成功连接界面如下。说明：左半区为 Windows 系统下的文件，右半区为 Linux 系统下的文件，可用拖动文件方式进行文件的跨平台传输



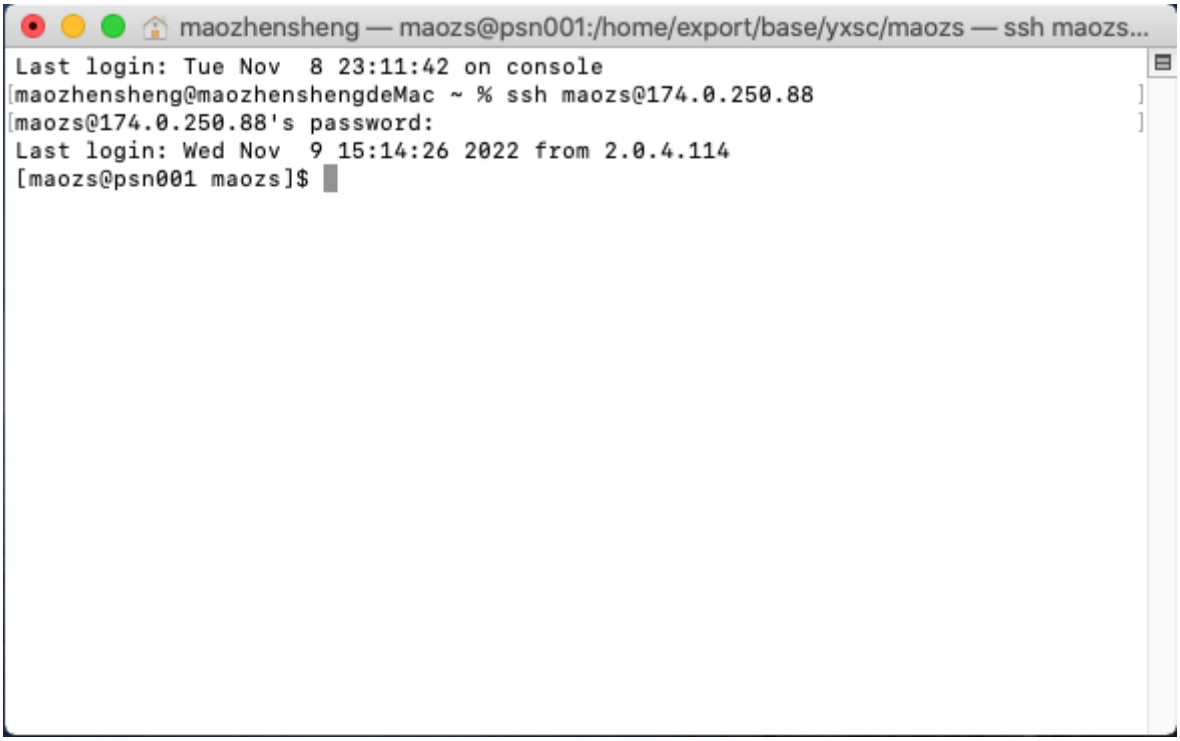
> **注**：建议将所有文件放入online1目录下，online1目录读写速度更快，空间更大！

2.2.2 MAC 用户

步骤一：macOS 连接超算

Mac 用户可以通过自带终端通过 ssh 命令连接盐城超算，具体命令为 ssh username@174.0.250.88，其中，username 为您开通的用户名

输入密码，登录成功（**输入密码时，不显示密码属于正常情况，输入完按回车即可**）



连接之后，建议进入 online1 目录再进行操作，home 目录下磁盘大小会进行限额，默认限额为 1GB，online1 目录磁盘免费容量为 4TB，如需扩充，请联系销售

步骤二：Mac 传文件

scp（secure copy）是一个基于 SSH 协议在网络之间进行安全传输的命令，其格式为 “scp [参数]本地文件 远程账户@远程IP地址:远程目录”。

在使用 scp 命令把文件从本地复制到远程主机时，首先需要以绝对路径的形式写清本地文件的存放位置。如果要传送整个文件夹内的所有数据，还需要额外添加参数 -r 进行递归操作。然后写上要传送到的远程主机的IP地址，远程服务器会要求进行身份验证，此时输入远程服务器的密码进行文件传输。

```
scp /root/readme.txt maozs@192.168.10.10:/home
```

此外，还可以使用 scp 命令把远程服务器上的文件下载到本地主机，其命令格式为 “scp [参数]远程用户@远程IP地址:远程文件 本地目录”。

```
scp maozs@192.168.10.10:/etc/redhat-release /root
```

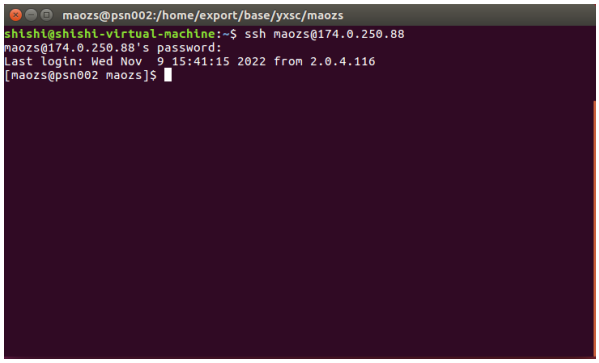
2.2.3 Linux用户

步骤一：Linux 连接超算

Linux 用户可以在终端中使用 ssh 命令登录，具体命令为：

```
ssh username@174.0.250.88   #username 为开通的用户名
```

输入密码



连接之后，建议进入 online1 目录再进行操作，home 目录下磁盘大小会进行限额，默认限额为 1GB，online1 目录磁盘免费容量为 4TB，如需扩充，请联系销售

> 注：输入密码不会显示，正常输入完然后回车即可

步骤二：Linux 传文件

scp 传输

scp（secure copy）是一个基于 SSH 协议在网络之间进行安全传输的命令，其格式为 " **scp [参数]本地文件 远程账户@远程IP地址:远程目录**"。

在使用 scp 命令把文件从本地复制到远程主机时，首先需要以绝对路径的形式写清本地文件的存放位置。如果要传送整个文件夹内的所有数据，还需要额外添加参数 -r 进行递归操作。然后写上要传送到的远程主机的IP地址，远程服务器便会要求进行身份验证，此时输入远程服务器的密码进行文件传输。

```
scp /root/readme.txt maozs@192.168.10.10:/home:
```

此外，还可以使用 scp 命令把远程服务器上的文件下载到本地主机，其命令格式为 "**scp [参数]远程用户@远程IP地址:远程文件 本地目录**"。

```
scp maozs@192.168.10.10:/etc/redhat-release /root
```

三 计算任务提交

3.1 编写任务提交脚本

新建文件`run.sh`，输入以下内容：

```
#!/bin/bash
#SBATCH -p q_amd_share
#SBATCH -n 2
#SBATCH -o job.out

mpirun -n 2 ./a.out
```

- `#!/bin/bash`，固定内容，不用修改
- `#SBATCH -p q_amd_share`，向调度系统申请`q_amd_share`队列的计算资源。超算有很多计算服务器，服务器划分成不同的队列管理使用。调整队列名，即更换队列运行计算任务。两大主要使用队列：
  - **q_amd_share分区，节点配置：AMD Epyc 7H12处理器，128核、256G内存**
  - **q_intel_share，节点配置：Intel Xeon 8358处理器，64核，256G内存**
- `#SBATCH -n 2`，申请2个核

- `#SBATCH -o job.out`，程序的运行输出保存在`job.out`文件，该文件文件名可以随意修改
- `mpirun -n 2 ./a.out`程序自身的运行命令

3.2 提交计算任务

执行`sbatch  run.sh`提交计算任务

```
[admin@psn003 tmp]$ sbatch run.sh
Submitted batch job 3656
```

3.3 查看任务状态

执行`squeue -u 用户名`查看计算任务运行状态

```
[addmin@psn003 tmp]$ squeue -u admin
             JOBID PARTITION     NAME     USER ST       TIME  NODES
NODELIST(REASON)
              3656 q_amd_sha   run.sh    admin  R       0:23     10 bn[054-063]
```

- JOBID，任务编号
- ST，任务状态，作业状态包括`R`(正在运行)，`PD`(正在排队)，`CG`(即将完成)，`CD`(已完成)
- TIME，运行时间
- NODES，占用节点数目（也称服务器，下同）个数
- NODELIST，占用服务器的编号

3.4 查看程序输出

计算任务运行起来后，执行`tail -f job.out`可 以查看程序的实时输出，执行`Ctrl+C`退出查看

```
[admin@psn003 tmp]$ tail -f job.out
Hello world from processor bn060, rank 896 out of 1280 processors
Hello world from processor bn060, rank 897 out of 1280 processors
```

3.4.1 查看分区计算资源使用情况

```
[admin@psn003 tmp]$ sinfo
PARTITION    AVAIL  TIMELIMIT  NODES  STATE NODELIST
q_amd_share*    up   infinite     43  alloc bn[023-050,334-336,345-356]
q_amd_share*    up   infinite     52   idle bn[053-096,337-344]
```

节点状态包括：

`drain`(节点故障)，`alloc`(节点在用)，`idle`(节点可用)，`down`(节点下线)，`mix`(节点部分占用，但仍有剩余资源)

3.4.2 计算任务停止

执行`scancel  任务编号`停止计算任务，“任务编号”通过`myjob`的`JOBID`字段可以查到。

```
scancel  3656
```

3.4.3 计算任务详细信息查看

执行 `scontrol show job 任务编号` 可以查到计算任务的工作目录、输出文件等详细信息。

```
scontrol show job 3644
```

四 进阶使用指南

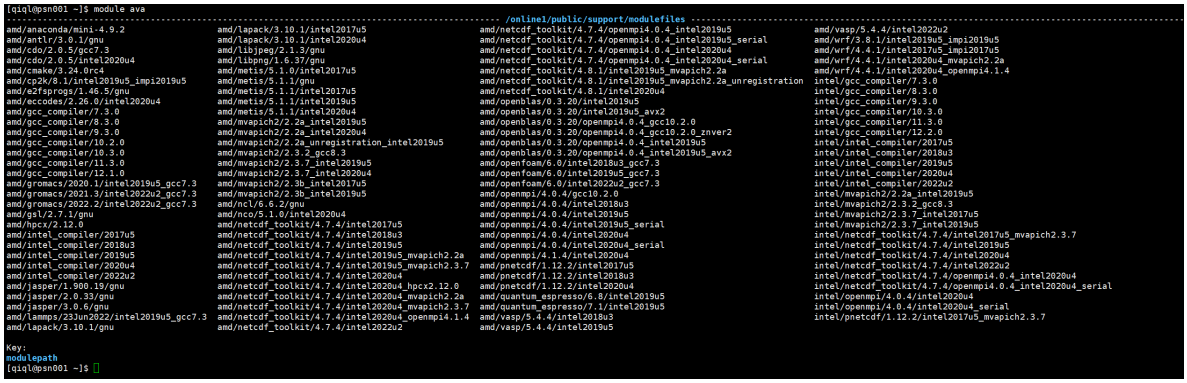
4.1 概要

1. 在通过 ssh 登录到集群之后，集群根据登录节点的负载情况让用户进入 3 台 登录节点（主机号范围为 psn[001-003] ）中的某一台。**登录之后先进入 online1 目录再操作**，否则作业将提交失败！用户可以在登录节点安装或编译程序，但尽量**不要在登录节点直接运行程序。在登录节点上用 mpirun 启用多个进程运行程序会报错，这属于正常情况**，您需要将程序提交到计算节点执行。
2. 程序或算例尽量放入到 **online1** 目录，home 目录磁盘限额为 **1GB**，文件数限额为 6万，`online1` 目录磁盘限额为 **4TB**，文件数限额为 60万。且 online1 目录采用 DDN 高速存储，相比于 home 目录，读写速度更快。
3. 集群使用 **module** 来管理软件环境，使用 `module avail` 可以查看集群中现有的软件，使用 `module load 模块名` 可以加载对应的软件环境，使用 `module unload 模板名` 对软件环境进行卸载。模块名的含义为 `编译程序使用的处理器 / 软件名 / 软件版本 [/ 编译软件所使用的编译器]` ，如 `amd/gromacs/2020.1/intel2019u5_gcc7.3` 表示在 AMD 处理器上使用 Intel2019update5 和 gcc7.3 编译器编译安装的 gromacs 软件，版本为 2020.1。
4. 使用 srun 可以在前台提交作业，但如果使用 srun 提交 **MPI** 程序，需要先执行 `export I_MPI_PMI_LIBRARY=/opt/hpc/slurm/21.08.6/lib/libpmi.so` 否则作业可能会提交失败；如果自行编写作业脚本，作业脚本中使用 mpirun 命令运行程序，那么在使用 sbatch 命令提交作业前，需要先执行 `unset I_MPI_PMI_LIBRARY` ，否则作业可能提交失败。
5. 提交作业后使用 `squeue` 查看作业队列，或使用 `myjob` 命令只查看自己的作业，使用 `bkill jobid` 取消作业，使用 `scancel -u username` 取消账户下的所有作业。

4.2 module

本集群使用 module 进行软件环境的管理，您只需输入短短的几行命令即可加载/卸载软件环境或更换同一个软件的不同版本

使用 `module avail` 命令查看本集群上已经安装好的全部软件，如下：



4.2.1 加载软件

使用 `module load 模块名` 加载您需要的软件环境。例如，加载 Intel 2017update5 编译器的命令为 `module load amd/intel_compiler/2017u5` ，如：

```
[qiql@psn001 ~]$ module load amd/intel_compiler/2017u5
[qiql@psn001 ~]$ icc -v
icc version 17.0.5 (gcc version 4.8.5 compatibility)
[qiql@psn001 ~]$ mpiicc -v
mpiicc for the Intel(R) MPI Library 2017 Update 4 for Linux*
Copyright(C) 2003-2017, Intel Corporation.  All rights reserved.
icc version 17.0.5 (gcc version 4.8.5 compatibility)
[qiql@psn001 ~]$
```

使用`module load 模块1 模块2 模块3`同时加载多个模块，如：

```
[qiql@psn001 ~]$ module load amd/intel_compiler/2022u2 amd/cmake/3.24.0rc4
[qiql@psn001 ~]$ module list
Currently Loaded Modulefiles:
 1) amd/intel_compiler/2022u2   2) amd/cmake/3.24.0rc4
[qiql@psn001 ~]$
```

> **注意**：使用 module 加载的环境仅临时存在于会话中，下次登录后仍需要重新加载，如需使环境永久生效，建议将 module 命令写入到 home 目录下的 `.bashrc` 文件中。

4.2.2 查看软件

使用 `module list` 命令查看当前会话中加载的全部模块，如：

```
[qiql@psn001 ~]$ module list
Currently Loaded Modulefiles:
 1) amd/intel_compiler/2022u2   2) amd/cmake/3.24.0rc4   3) amd/intel_compiler/2017u5   4) amd/netcdf_toolkit/4.7.4/intel2017u5   5) amd/wrf/4.4.1/intel2017u5_impi2017u5

Key:
auto-loaded
[qiql@psn001 ~]$
```

4.2.3 卸载软件

使用 `module unload 模块名` 卸载相应的环境。例如，您需要卸载 Intel 2017update5 并行编译器，则命令为：`module unload amd/intel_compiler/2017u5`

如果您需要将会话中的环境全部卸载，可以使用命令：`module purge`

4.3 切换登录节点

如当前节点内存被消耗较多感到卡顿，可以切换登录节点。切换命令为 ssh hostname，如：

```
[maozs@psn002 ~]$ ssh psn001
maozs@psn001's password:
Last failed login: Wed Nov  9 15:41:46 CST 2022 from 174.0.250.2 on ssh:notty
There was 1 failed login attempt since the last successful login.
Last login: Wed Nov  9 15:34:19 2022 from 2.0.4.116
[maozs@psn001 maozs]$ ssh psn003
maozs@psn003's password:
Last login: Wed Nov  9 15:16:42 2022 from 2.0.4.114
[maozs@psn003 maozs]$
```

4.4 slurm 介绍

盐城超算平台采用 slurm 作业调度系统来对计算资源进行统一管理和调配，slurm 被广泛用于世界上大部分的超级计算机中。

slurm 基本命令：

Slurm 命令	功能
sinfo	查看集群队列使用情况
squeue	查看作业队列
sbatch	后台提交作业
srun	前台提交作业
scontrol	查看和修改作业参数
sacct	已完成作业报告
scancel	删除已提交的作业

提交作业时，可以采用前台提交（srun），后台提交（sbatch）两种方式

4.4.1 查看集群队列情况

终端执行 `sinfo`，查看集群队列情况：

```
[qiql@psn001 ~]$ sinfo
PARTITION        AVAIL  TIMELIMIT  NODES  STATE NODELIST
q_amd_share*        up   infinite      1    mix bn362
q_amd_share*        up   infinite     38  alloc bn[001-012,071-090,363,367-371]
q_amd_share*        up   infinite    356   idle bn[013-017,023-070,091-361,364-366,372-400]
q_amd_gpu_nvidia    up   infinite     10   idle cn[039-048]
q_intel_share       up   infinite      2    mix dn[002,004]
q_intel_share       up   infinite     57   idle dn[003,005-060]
[qiql@psn001 ~]$
```

- PARTITION 为队列分区，目前队列开放的有 q_amd_share, q_intel_share, q_amd_gpu_nvidia
- AVAIL 为可用情况，up 表示队列可用
- TIMELIMIT 为作业运行时长上限，infinite 表示该队列不限制作业运行时间
- NODES 为该分区的节点数目
- STATE 为该分区的状态，分为：`drain`(节点故障)，`alloc`(节点在用)，`idle`(节点可用)，`down`(节点下线)，`mix`(节点部分占用，但仍有剩余资源)
- NODELIST 为该分区的节点主机号范围

4.4.2 查看作业队列

终端执行 `squeue`，查看集群当前作业的队列

```
[qiql@psn001 root]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
              5595 q_amd_sha   sf_B70     test  R 39-20:07:52      1 bn367
             10767 q_amd_sha test_tai     john  R 5-23:58:48     12 bn[001-012]
             10774 q_amd_sha test_tai    candy  R 5-23:53:53     20 bn[071-090]
             11061 q_amd_sha lht_elan      tom  R 2-06:53:48      1 bn362
             11062 q_amd_sha lht_elan    kitty  R 2-06:53:32      1 bn363
             11064 q_amd_sha lht_elan    james  R 2-06:48:13      1 bn363
             11267 q_amd_sha  tc_3A1C   harden  R    5:27:40      4 bn[368-371]
             11155 q_intel_s NV_AFNON   howard  R 1-07:50:25      1 dn004
             11285 q_intel_s MetNet2_     kobe  R    1:42:27      1 dn004
             11286 q_intel_s ConvLstm   iverson  R    1:40:12      1 dn002
[qiql@psn001 root]$
```

- JOBID 为作业号
- PARTITION 为该作业所在的分区
- name 为作业名
- USER 为提交该作业的用户名
- ST 为该作业的运行状态，R 为正在运行，PD 为正在排队，CG 为即将完成
- TIME 为作业运行的时间

- NODES 为作业使用的节点数目
- NODELIST 为该作业使用的节点主机范围

4.5 前台提交作业

- 作业分为前台提交和后台提交，前台提交作业时，如果会话中断，那么作业也会停止运行，适合用来调试程序，如果希望程序能一直在后台运行，请编写作业脚本然后使用 `sbatch` 命令进行作业提交。
- 请在 online1 目录下提交作业，**在 home 下提交作业会失败**。online1 目录免费额度为4TB，文件数免费额度为 60 万，如需扩充存储，请联系销售。

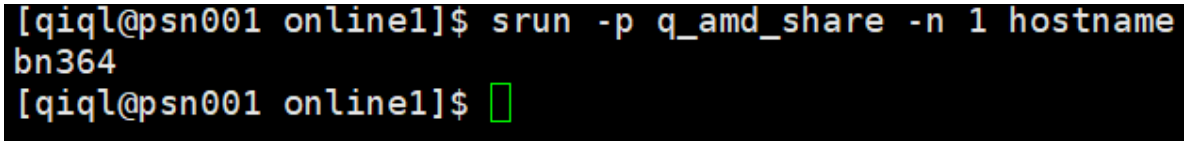
前台提交作业命令为 srun，参数如下：

```
-p, --partion=q_amd_share        #指定分区
-J, --job-name=JOBNAME           #指定作业名称
-N, --nodes=N                    #指定程序使用节点数
-n, --ntask=NTASKs               #指定程序的总进程数
-G, --gpus=N                     #指定程序使用GPU数【可选】
--gpus-per-task=1                #指定每个任务所使用的GPU数【可选】
-o, --output=out_filename        #指定输出文件输出
-e, --error=error_filename       #指定错误文件输出
--ntask-per-node=N               #指定每个节点进程数/核数【可选】
-c, --cpu-per-task=NCPUs         #指定每个进程所使用的 CPU 核数，不指定默认为1 【可选】
-w, --nodelist=node[1,2]         #指定使用主机号【可选】
--exclusive                      #独占节点；注意：假如程序实际上只用了 1 个节点的 1 个核心，开启独占后，将按该节点的总核心数进行计费【可选】
```

4.5.1 示例 1 非 MPI 程序

```
srun -p q_amd_share -n 1 hostname
```

该命令将向队列 q_amd_share 申请 1 个 CPU 核心，运行 hostname 命令，如图：



4.5.2 示例 2 单节点或跨节点 MPI 程序

假如要提交 MPI 程序，需提前加载 MPI 编译器环境，然后自行编译之后再进行提交。

使用 srun 提交之前，先执行命令 `export I_MPI_PMI_LIBRARY=/opt/hpc/slurm/21.08.6/lib/libpmi.so`

```
srun -p q_amd_share -n 4 ./mpi_hello
```

该命令将向 q_amd_share 队列申请 4 个 CPU 核心运行 mpi_hello 程序，由于未使用 -N 参数指定节点数且 CPU 核心数小于一个节点的总核心数（q_amd_share队列一个节点128核心），故默认使用 1 个节点

```
[qiql@psn001 online1]$ module purge
[qiql@psn001 online1]$ module load amd/intel_compiler/2022u2
[qiql@psn001 online1]$ mpiicc -v
mpiicc for the Intel(R) MPI Library 2021.6 for Linux*
Copyright Intel Corporation.
icc version 2021.6.0 (gcc version 4.8.5 compatibility)
[qiql@psn001 online1]$ module list
Currently Loaded Modulefiles:
 1) amd/intel_compiler/2022u2
[qiql@psn001 online1]$ mpiicc -o mpi_hello ./mpi_hello.c
[qiql@psn001 online1]$ export I_MPI_PMI_LIBRARY=/opt/hpc/slurm/21.08.6/lib/libpmi.so
[qiql@psn001 online1]$ srun -p q_amd_share -n 4 ./mpi_hello
Hello world from processor bn364, rank 0 out of 4 processors
Hello world from processor bn364, rank 2 out of 4 processors
Hello world from processor bn364, rank 3 out of 4 processors
Hello world from processor bn364, rank 1 out of 4 processors
[qiql@psn001 online1]$
```

假如要跨节点运行 MPI 程序，需指定 -N 参数，假如希望独占这些节点，需加上 --exclusive 参数，假如需要指定所使用的节点，需添加 --nodelist 参数，如：

```
srun -p q_amd_share -N 10 -n 10 --nodelist=bn[013-015,017,023-028] --exclusive
./mpi_hello
```

> **注意**：1. 开启独占后，即使这些节点仍有空闲核心，但其他用户也无法向这些节点提交作业
>
> 2. 开启独占将按节点核心总数进行计算核时费，而非按照程序实际使用的核心数目进行计费

该命令将向 q_amd_share 队列申请 bn[013-015,017,023-028] 这 10 个节点运行 mpi_hello 程序，且该作业独占这 10 个节点，-n 10 指定了运行该程序的总进程数，这 10 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 1 个进程，如图：

```
[qiql@psn001 online1]$ srun -p q_amd_share -N 10 -n 10 --nodelist=bn[013-015,017,023-028] --exclusive  ./mpi_hello
Hello world from processor bn027, rank 8 out of 10 processors
Hello world from processor bn025, rank 6 out of 10 processors
Hello world from processor bn026, rank 7 out of 10 processors
Hello world from processor bn023, rank 4 out of 10 processors
Hello world from processor bn028, rank 9 out of 10 processors
Hello world from processor bn024, rank 5 out of 10 processors
Hello world from processor bn013, rank 0 out of 10 processors
Hello world from processor bn014, rank 1 out of 10 processors
Hello world from processor bn017, rank 3 out of 10 processors
Hello world from processor bn015, rank 2 out of 10 processors
[qiql@psn001 online1]$
```

4.5.3 示例3 混合 MPI / openMP 程序

假如程序中的并行模式为 MPI + openMP，那么需要定义环境变量 **OMP_NUM_THREADS** 来控制每个进程所开启的线程数

```
export OMP_NUM_THREADS=3 # 每个进程开启 3 个线程

srun -p q_amd_share -N 2 -n 4 --nodelist=bn[013-014] --ntasks-per-node=2 --cpus-
per-task=3  ./mpi_openmp_hello
```

> 环境变量 **OMP_NUM_THREADS** 用来指定一个进程的线程数目，此环境变量**不一定生效**，如果程序中已指定了线程数目，那么将**覆盖**此环境变量所设置的值
>
> tips: openMP 程序中不指定线程数目的方法：
> 代码中将指令 **# pragma omp parallel num_threads(threads_count)** 替换为：**# pragma omp parallel** 即可

该命令将向 q_amd_share 队列申请 bn013 和 bn014 这 2 个节点运行 mpi_openmp_hello 程序，其中：

-N 2 指定了运行该程序的总节点数；

-nodelist=bn[013-014] 指定了运行程序的节点号范围为 bn013 和 bn014

-n 4 指定了运行该程序的总进程数；

--ntasks-per-node=2 指定了每个节点的进程数；

--cpus-per-task=3 指定了每个进程使用的 CPU 核心数，若不指定该参数，默认为 1，即每个进程默认使用 1 个 CPU 核心；

在此示例中，每个节点将运行 2 个进程，每个进程运行 3 个线程，共使用 12 个CPU 核心，如图：

```
[qiql@psn001 guide]$ srun -p q_amd_share -N 2 -n 4 --nodelist=bn[023-024]   --ntasks-per-node=2 --cpus-per-task=6  ./mpi_openmp_hello
Hello world from thread 0 ,thread num: 3 ; processor bn023, rank 0 out of 4 processors
Hello world from thread 2 ,thread num: 3 ; processor bn023, rank 0 out of 4 processors
Hello world from thread 1 ,thread num: 3 ; processor bn023, rank 0 out of 4 processors
Hello world from thread 0 ,thread num: 3 ; processor bn023, rank 1 out of 4 processors
Hello world from thread 2 ,thread num: 3 ; processor bn023, rank 1 out of 4 processors
Hello world from thread 1 ,thread num: 3 ; processor bn023, rank 1 out of 4 processors
Hello world from thread 0 ,thread num: 3 ; processor bn024, rank 2 out of 4 processors
Hello world from thread 1 ,thread num: 3 ; processor bn024, rank 2 out of 4 processors
Hello world from thread 2 ,thread num: 3 ; processor bn024, rank 2 out of 4 processors
Hello world from thread 0 ,thread num: 3 ; processor bn024, rank 3 out of 4 processors
Hello world from thread 1 ,thread num: 3 ; processor bn024, rank 3 out of 4 processors
Hello world from thread 2 ,thread num: 3 ; processor bn024, rank 3 out of 4 processors
[qiql@psn001 guide]$
```

4.5.4 示例4 单节点 GPU 程序

目前每个 GPU 节点上有 8 张卡

假如作业使用的卡数量小于 8 ，即只使用1 个节点，那么srun提交命令为：

```
module load amd/cuda/11.8.89 # 加载 cuda11.8.89 环境

srun -p q_intel_gpu_nvidia --gres=gpu:1 ./hello_world
```

该命令将向 q_intel_gpu_nvidia 队列申请 1 张卡运行 hello_world 程序

```
[test001@psn001 online1]$ srun -p q_intel_gpu_nvidia  -n 1 --gpus-per-task=1 hello_world
Hello World from host!
Hello World! from thread [0,0]          From device
```

4.6 后台提交作业

如希望断开会话连接后作业仍在运行，建议采用 `sbatch` 进行后台提交作业

后台提交需要用户先编写作业脚本，指定一些参数来申请程序所使用的节点数，进程数，线程数等信息。脚本名称无限制（一般为 run.sh），编写完后，使用 **sbatch 脚本名** 提交作业。脚本第一行默认为 `#!/bin/bash` ，请勿随意更改。

以下为脚本示例，您可以根据您的需要进行参数的删除或修改

```
#!/bin/bash
#SBATCH --job-name=JOBNAME       #指定作业名称
#SBATCH -p q_amd_share           #指定分区
#SBATCH -N 2                     #指定节点数量
#SBATCH -n 32                    #指定总进程数
#SBATCH -G 8                     #指定程序使用GPU数【可选】
#SBATCH --gpus-per-task=1        #指定每个任务所使用的GPU数【可选】
#SBATCH -o job.out               #指定输出文件输出
#SBATCH -e job.error.out         #指定错误文件输出
#SBATCH --ntasks-per-node=16     #指定每个节点进程数【可选】
#SBATCH --cpus-per-task=1        #指定每个进程使用 CPU 核心数，不指定默认为 1 【可选】
#SBATCH --nodelist=node[3,4]     #指定优先使用节点【可选】
#SBATCH --exclude=node[1,5-6]    #指定避免使用节点【可选】
#SBATCH --exclusive              #独占该节点【可选】
```

```
unset I_MPI_PMI_LIBRARY          # 使用mpirun运行程序，需卸载此环境变量

module load 模块名               #加载软件或其他环境变量
mpirun -n 32 test                #作业运行命令
```

4.6.1 示例 1 非 MPI 程序

编写作业脚本 run.sh（脚本名名称无限制）

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH -p q_amd_share                      # 指定作业提交的分区
#SBATCH -N 1                                # 指定节点数量为 1
#SBATCH -n 1                                # 指定总进程数为 2
#SBACTH -o out                              # 指定作业输出文件名 %j 是作业号
#SBATCH -e %j_.error.out                    # 指定作业中的错误所输出的文件名，%j 是作业
号
#SBATCH --ntasks-per-node=1                 # 每个节点开启 1 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                   # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn114                    # 指定所使用的的节点【可选】
#SBATCH --exclusive                         # 使该作业独占该节点【可选】

hostname                                    # 作业运行命令
```

执行 `sbatch run.sh` 进行作业提交

该作业将向 q_amd_share 队列的 bn114 节点申请 1 个 CPU 核心，且独占该节点，程序使用 1 个进程，每个进程只开启 1 个线程

4.6.2 示例 2 单节点或跨节点 MPI 程序

编写作业脚本 run.sh（脚本名称无限制）

> **注意：使用 mpirun 运行程序时，需要在 mpirun 命令前执行 unset I_MPI_PMI_LIBRARY**

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH -p q_amd_share                      # 指定作业提交的分区
#SBATCH -N 1                                # 指定节点数量为 1
#SBATCH -n 2                                # 指定总进程数为 2
#SBACTH -o job.%j.out                       # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                    # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH --ntasks-per-node=2                 # 每个节点开启 2 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                   # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn114                    # 指定所使用的的节点【可选】
#SBATCH --exclusive                         # 使该作业独占该节点【可选】

unset I_MPI_PMI_LIBRARY                     # 使用 mpirun 运行程序，需卸载此环境变量
module load amd/intel_compiler/2019u5       # 加载环境
mpiicc -o mpi_hello ./mpi_hello.c           # 编译程序
mpirun -n 2 ./mpi_hello                     # 运行程序
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_amd_share 队列申请 1 个节点运行程序，指定节点为 bn114 且独占该节点

假如MPI程序需要多节点并行，修改作业脚本为：

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH --partition=q_amd_share             # 指定作业提交的分区
#SBATCH -N 10                               # 指定节点数量为 10
#SBATCH -n 20                               # 指定总进程数为 20
#SBACTH -o job.%j.out                       # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                    # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH --ntasks-per-node=2                 # 每个节点开启 2 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                   # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn[013-015,017,023-028] # 指定所使用的的节点【可选】
#SBATCH --exclusive                         # 使该作业独占该节点【可选】

unset I_MPI_PMI_LIBRARY                     # 使用 mpirun 运行程序时，需卸载此环境变量
module load amd/intel_compiler/2019u5       # 加载环境
mpiicc -o mpi_hello ./mpi_hello.c           # 编译程序
mpirun -n 20 ./mpi_hello                    # 运行程序
```

该作业脚本将向 q_amd_share 队列申请 bn[013-015,017,023-028] 这 10 个节点运行 mpi_hello 程序，且该作业独占这 10 个节点，-n 10 指定了运行该程序的总进程数，这 10 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 1 个进程。

4.6.3 示例 3 混合 MPI / openMP 程序

假如程序需要用到 **openMP** 开启多线程并行，需要在作业脚本中使用 **OMP_NUM_THREADS** 环境变量来指定线程数目。

编写作业脚本 `run.sh`（脚本名称无限制）

> **注意：使用 mpirun 运行程序时，需要在 mpirun 命令前执行** `unset I_MPI_PMI_LIBRARY`

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH --partition=q_amd_share             # 指定作业提交的分区
#SBATCH -N 2                                # 使用 2 个节点
#SBATCH --nodelist=bn[013-014]              # 指定所使用的的节点
#SBATCH -o %j.out                           # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                    # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH -n 4                                # 指定进程总数为 4
#SBATCH --ntasks-per-node=2                 # 每个节点使用 2 个进程，不指定时，slurm 会
尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=3                   # 每个进程使用 3 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --exclusive                         # 使该作业独占该节点【可选】

export OMP_NUM_THREADS=3                    # 每个进程开启 3 个线程（不一定生效，如果程序
中指定了线程数量，将覆盖此环境变量）
unset I_MPI_PMI_LIBRARY                     # 使用 mpirun 运行程序时，需先卸载此环境变量
module load amd/intel_compiler/2019u5       # 加载环境
mpiicc -qopenmp -o mpi_openmp_hello ./mpi_openmp_hello.c  # 编译程序命令
mpirun -n 4 ./mpi_openmp_hello            # 程序运行命令
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_amd_share 队列申请 bn[013-014] 这 2 个节点运行 mpi_openmp_hello 程序，且该作业独占这 2 个节点，该作业运行完毕之前，其他用户无法向这 2 个节点提交作业，-n 4 指定了运行该程序的进程总数，这 4 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 2 个进程，每个进程将开启 3 个 openMP 线程，此作业将申请到 4*3=12 个 CPU 核心。

> 假如程序中指定了线程数，**OMP_NUM_THREADS** 环境变量将不会生效。
>
> 尽量使线程数接近每个进程申请的 CPU 核心数，即：**--cpus-per-task** 所指定的参数等于 **OMP_NUM_THREADS** 的值，如不指定 **--cpus-per-task** 参数的值，每个进程将只会使用一个 CPU 核心，不利于多线程作业的执行。

4.6.4 示例4 单节点GPU程序

```
#!/bin/bash
#SBATCH --job-name=test1                 # 指定作业名
#SBATCH -o result.out                    # 指定输出文件名;默认为slurm-[jobID].out
#SBATCH -e error.out                     # 指定程序错误输出文件名
#SBATCH --partition=q_intel_gpu_nvidia   # 指定作业提交的分区
#SBATCH --gres=gpu:1                     # 指定作业使用卡的总数为 1

# 加载cuda模块
module load amd/cuda/11.8.89

# 使用nvcc编译cuda脚本
nvcc -o test1 test1.cu

## Run the script
./test1
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_intel_gpu_nvidia 队列申请使用 1 张 gpu 卡运行程序，不指定节点数默认在单节点运行

4.7 查看作业

使用 `squeue` 查看当前集群的作业队列

使用 `myjob` 可以只查看自己的作业

4.7 取消作业

取消作业的命令为 `scancel jobid`，或使用 `bkill jobid`

如需取消账户的所有作业，使用 `scancel -u username`

4.8 查找历史作业

使用 `sacct` 命令可以查看历史作业情况

示例：

```
sacct -S 2022-10-15T00:00:00 -o
"jobid,jobname,partition,user,alloccpus,state,nodelist,start,end,workdir%100" -X
```

该命令将查询从 2022 年 10 月 15 日 0 点以后的所有作业的作业号、作业名，所提交队列、账户名、申请的 CPU 核心数、作业状态、所使用的节点主机号、作业开始时间、作业结束时间、作业的提交目录（最长显示 100 个字符）。

如需进行二次模糊查找，可以使用管道符和 grep 进行筛选过滤。

执行 `scontrol show job jobid` 可以查看对应 `jobid` 的作业详情，如工作目录，运行时间等等

-

4.5 前台提交作业

- 作业分为前台提交和后台提交，前台提交作业时，如果会话中断，那么作业也会停止运行，适合用来调试程序，如果希望程序能一直在后台运行，请编写作业脚本然后使用 `sbatch` 命令进行作业提交。
- 请在 online1 目录下提交作业，**在 home 下提交作业会失败。**online1 目录免费额度为4TB，文件数免费额度为 60 万，如需扩充存储，请联系销售。

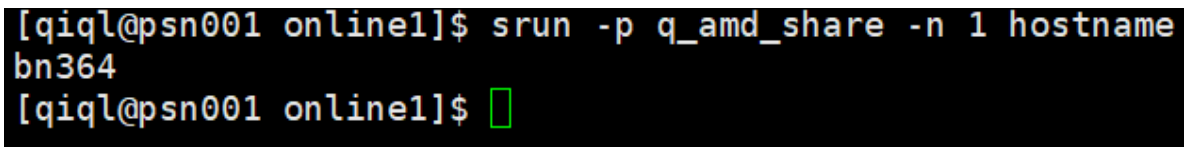
前台提交作业命令为 srun，参数如下：

```
-p, --partion=q_amd_share        #指定分区
-J, --job-name=JOBNAME           #指定作业名称
-N, --nodes=N                    #指定程序使用节点数
-n, --ntask=NTASKs               #指定程序的总进程数
-G, --gpus=N                     #指定程序使用GPU数【可选】
--gpus-per-task=1                #指定每个任务所使用的GPU数【可选】
-o, --output=out_filename        #指定输出文件输出
-e, --error=error_filename       #指定错误文件输出
--ntask-per-node=N               #指定每个节点进程数/核数【可选】
-c, --cpu-per-task=NCPUs         #指定每个进程所使用的 CPU 核数，不指定默认为1 【可选】
-w, --nodelist=node[1,2]         #指定使用主机号【可选】
--exclusive                      #独占节点；注意：假如程序实际上只用了 1 个节点的 1 个核心，开启独占后，将按该节点的总核心数进行计费【可选】
```

4.5.1 示例 1 非 MPI 程序

```
srun -p q_amd_share -n 1 hostname
```

该命令将向队列 q_amd_share 申请 1 个 CPU 核心，运行 hostname 命令，如图：



4.5.2 示例 2 单节点或跨节点 MPI 程序

假如要提交 MPI 程序，需提前加载 MPI 编译器环境，然后自行编译之后再进行提交。

使用 srun 提交之前，先执行命令 `export I_MPI_PMI_LIBRARY=/opt/hpc/slurm/21.08.6/lib/libpmi.so`

```
srun -p q_amd_share -n 4 ./mpi_hello
```

该命令将向 q_amd_share 队列申请 4 个 CPU 核心运行 mpi_hello 程序，由于未使用 -N 参数指定节点数且 CPU 核心数小于一个节点的总核心数（q_amd_share队列一个节点128核心），故默认使用 1 个节点

```
[qiql@psn001 online1]$ module purge
[qiql@psn001 online1]$ module load amd/intel_compiler/2022u2
[qiql@psn001 online1]$ mpiicc -v
mpiicc for the Intel(R) MPI Library 2021.6 for Linux*
Copyright Intel Corporation.
icc version 2021.6.0 (gcc version 4.8.5 compatibility)
[qiql@psn001 online1]$ module list
Currently Loaded Modulefiles:
 1) amd/intel_compiler/2022u2
[qiql@psn001 online1]$ mpiicc -o mpi_hello ./mpi_hello.c
[qiql@psn001 online1]$ export I_MPI_PMI_LIBRARY=/opt/hpc/slurm/21.08.6/lib/libpmi.so
[qiql@psn001 online1]$ srun -p q_amd_share -n 4 ./mpi_hello
Hello world from processor bn364, rank 0 out of 4 processors
Hello world from processor bn364, rank 2 out of 4 processors
Hello world from processor bn364, rank 3 out of 4 processors
Hello world from processor bn364, rank 1 out of 4 processors
[qiql@psn001 online1]$
```

假如要跨节点运行 MPI 程序，需指定 -N 参数，假如希望独占这些节点，需加上 --exclusive 参数，假如需要指定所使用的节点，需添加 --nodelist 参数，如：

```
srun -p q_amd_share -N 10 -n 10 --nodelist=bn[013-015,017,023-028] --exclusive
./mpi_hello
```

> **注意**：1. 开启独占后，即使这些节点仍有空闲核心，但其他用户也无法向这些节点提交作业
>
> 2. 开启独占将按节点核心总数进行计算核时费，而非按照程序实际使用的核心数目进行计费

该命令将向 q_amd_share 队列申请 bn[013-015,017,023-028] 这 10 个节点运行 mpi_hello 程序，且该作业独占这 10 个节点，-n 10 指定了运行该程序的总进程数，这 10 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 1 个进程，如图：

```
[qiql@psn001 online1]$ srun -p q_amd_share -N 10 -n 10 --nodelist=bn[013-015,017,023-028] --exclusive  ./mpi_hello
Hello world from processor bn027, rank 8 out of 10 processors
Hello world from processor bn025, rank 6 out of 10 processors
Hello world from processor bn026, rank 7 out of 10 processors
Hello world from processor bn023, rank 4 out of 10 processors
Hello world from processor bn028, rank 9 out of 10 processors
Hello world from processor bn024, rank 5 out of 10 processors
Hello world from processor bn013, rank 0 out of 10 processors
Hello world from processor bn014, rank 1 out of 10 processors
Hello world from processor bn017, rank 3 out of 10 processors
Hello world from processor bn015, rank 2 out of 10 processors
[qiql@psn001 online1]$
```

4.5.3 示例3 混合 MPI / openMP 程序

假如程序中的并行模式为 MPI + openMP，那么需要定义环境变量 **OMP_NUM_THREADS** 来控制每个进程所开启的线程数

```
export OMP_NUM_THREADS=3 # 每个进程开启 3 个线程

srun -p q_amd_share -N 2 -n 4 --nodelist=bn[013-014] --ntasks-per-node=2 --cpus-
per-task=3  ./mpi_openmp_hello
```

> 环境变量 **OMP_NUM_THREADS** 用来指定一个进程的线程数目，此环境变量**不一定生效**，如果程序中已指定了线程数目，那么将**覆盖**此环境变量所设置的值
>
> tips: openMP 程序中不指定线程数目的方法：
> 代码中将指令 **# pragma omp parallel num_threads(threads_count)** 替换为：**# pragma omp parallel** 即可

该命令将向 q_amd_share 队列申请 bn013 和 bn014 这 2 个节点运行 mpi_openmp_hello 程序，其中：

-N 2 指定了运行该程序的总节点数；

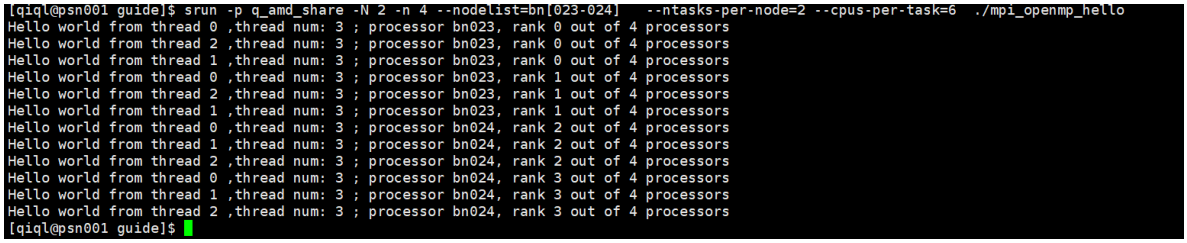
-nodelist=bn[013-014] 指定了运行程序的节点号范围为 bn013 和 bn014

-n 4 指定了运行该程序的总进程数；

--ntasks-per-node=2 指定了每个节点的进程数；

--cpus-per-task=3 指定了每个进程使用的 CPU 核心数，若不指定该参数，默认为 1，即每个进程默认使用 1 个 CPU 核心；

在此示例中，每个节点将运行 2 个进程，每个进程运行 3 个线程，共使用 12 个CPU 核心，如图：



4.5.4 示例4 单节点 GPU 程序

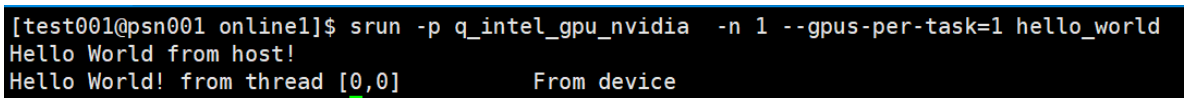
目前每个 GPU 节点上有 8 张卡

假如作业使用的卡数量小于 8 ，即只使用1 个节点，那么srun提交命令为：

```
module load amd/cuda/11.8.89 # 加载 cuda11.8.89 环境

srun -p q_intel_gpu_nvidia --gres=gpu:1 ./hello_world
```

该命令将向 q_intel_gpu_nvidia 队列申请 1 张卡运行 hello_world 程序



4.6 后台提交作业

如希望断开会话连接后作业仍在运行，建议采用 `sbatch` 进行后台提交作业

后台提交需要用户先编写作业脚本，指定一些参数来申请程序所使用的节点数，进程数，线程数等信息。脚本名称无限制（一般为 run.sh），编写完后，使用 **sbatch 脚本名** 提交作业。脚本第一行默认为 `#!/bin/bash` ，请勿随意更改。

以下为脚本示例，您可以根据您的需要进行参数的删除或修改

```
#!/bin/bash
#SBATCH --job-name=JOBNAME      #指定作业名称
#SBATCH -p q_amd_share          #指定分区
#SBATCH -N 2                    #指定节点数量
#SBATCH -n 32                   #指定总进程数
#SBATCH -G 8                    #指定程序使用GPU数【可选】
#SBATCH --gpus-per-task=1       #指定每个任务所使用的GPU数【可选】
#SBATCH -o job.out              #指定输出文件输出
#SBATCH -e job.error.out        #指定错误文件输出
#SBATCH --ntasks-per-node=16    #指定每个节点进程数【可选】
#SBATCH --cpus-per-task=1       #指定每个进程使用 CPU 核心数，不指定默认为 1 【可选】
```

```
#SBATCH --nodelist=node[3,4]     #指定优先使用节点【可选】
#SBATCH --exclude=node[1,5-6]    #指定避免使用节点【可选】
#SBATCH --exclusive              #独占该节点【可选】

unset I_MPI_PMI_LIBRARY          # 使用mpirun运行程序，需卸载此环境变量

module load 模块名               #加载软件或其他环境变量
mpirun -n 32 test                #作业运行命令
```

4.6.1 示例 1 非 MPI 程序

编写作业脚本 run.sh（脚本名名称无限制）

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH -p q_amd_share                      # 指定作业提交的分区
#SBATCH -N 1                                # 指定节点数量为 1
#SBATCH -n 1                                # 指定总进程数为 2
#SBACTH -o out                              # 指定作业输出文件名 %j 是作业号
#SBATCH -e %j_.error.out                    # 指定作业中的错误所输出的文件名，%j 是作业
号
#SBATCH --ntasks-per-node=1                 # 每个节点开启 1 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                   # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn114                    # 指定所使用的的节点【可选】
#SBATCH --exclusive                         # 使该作业独占该节点【可选】

hostname                                    # 作业运行命令
```

执行 `sbatch run.sh` 进行作业提交

该作业将向 q_amd_share 队列的 bn114 节点申请 1 个 CPU 核心，且独占该节点，程序使用 1 个进程，每个进程只开启 1 个线程

4.6.2 示例 2 单节点或跨节点 MPI 程序

编写作业脚本 run.sh（脚本名称无限制）

> **注意：使用 mpirun 运行程序时，需要在 mpirun 命令前执行 unset I_MPI_PMI_LIBRARY**

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH -p q_amd_share                      # 指定作业提交的分区
#SBATCH -N 1                                # 指定节点数量为 1
#SBATCH -n 2                                # 指定总进程数为 2
#SBACTH -o job.%j.out                       # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                    # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH --ntasks-per-node=2                 # 每个节点开启 2 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                   # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn114                    # 指定所使用的的节点【可选】
#SBATCH --exclusive                         # 使该作业独占该节点【可选】

unset I_MPI_PMI_LIBRARY                     # 使用 mpirun 运行程序，需卸载此环境变量
module load amd/intel_compiler/2019u5       # 加载环境
```

```
mpiicc -o mpi_hello ./mpi_hello.c          # 编译程序
mpirun -n 2 ./mpi_hello                    # 运行程序
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_amd_share 队列申请 1 个节点运行程序，指定节点为 bn114 且独占该节点

假如MPI程序需要多节点并行，修改作业脚本为：

```
#!/bin/bash
#SBATCH --job-name=mpi_hello               # 指定作业名
#SBATCH --partition=q_amd_share            # 指定作业提交的分区
#SBATCH -N 10                              # 指定节点数量为 10
#SBATCH -n 20                              # 指定总进程数为 20
#SBACTH -o job.%j.out                      # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                   # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH --ntasks-per-node=2                # 每个节点开启 2 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                  # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn[013-015,017,023-028] # 指定所使用的的节点【可选】
#SBATCH --exclusive                        # 使该作业独占该节点【可选】

unset I_MPI_PMI_LIBRARY                    # 使用 mpirun 运行程序时，需卸载此环境变量
module load amd/intel_compiler/2019u5      # 加载环境
mpiicc -o mpi_hello ./mpi_hello.c          # 编译程序
mpirun -n 20 ./mpi_hello                   # 运行程序
```

该作业脚本将向 q_amd_share 队列申请 bn[013-015,017,023-028] 这 10 个节点运行 mpi_hello 程序，且该作业独占这 10 个节点，-n 10 指定了运行该程序的总进程数，这 10 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 1 个进程。

4.6.3 示例 3 混合 MPI / openMP 程序

假如程序需要用到 **openMP** 开启多线程并行，需要在作业脚本中使用 **OMP_NUM_THREADS** 环境变量来指定线程数目。

编写作业脚本 `run.sh`（脚本名称无限制）

> **注意：使用 mpirun 运行程序时，需要在 mpirun 命令前执行** `unset I_MPI_PMI_LIBRARY`

```
#!/bin/bash
#SBATCH --job-name=mpi_hello               # 指定作业名
#SBATCH --partition=q_amd_share            # 指定作业提交的分区
#SBATCH -N 2                               # 使用 2 个节点
#SBATCH --nodelist=bn[013-014]             # 指定所使用的的节点
#SBATCH -o %j.out                          # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                   # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH -n 4                               # 指定进程总数为 4
#SBATCH --ntasks-per-node=2                # 每个节点使用 2 个进程，不指定时，slurm 会
尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=3                  # 每个进程使用 3 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --exclusive                        # 使该作业独占该节点【可选】

export OMP_NUM_THREADS=3                   # 每个进程开启 3 个线程（不一定生效，如果程序
中指定了线程数量，将覆盖此环境变量）
```

```
unset I_MPI_PMI_LIBRARY                    # 使用 mpirun 运行程序时，需先卸载此环境变量
module load amd/intel_compiler/2019u5      # 加载环境
mpiicc -qopenmp -o mpi_openmp_hello ./mpi_openmp_hello.c  # 编译程序命令
mpirun -n 4 ./mpi_openmp_hello             # 程序运行命令
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_amd_share 队列申请 bn[013-014] 这 2 个节点运行 mpi_openmp_hello 程序，且该作业独占这 2 个节点，该作业运行完毕之前，其他用户无法向这 2 个节点提交作业，-n 4 指定了运行该程序的进程总数，这 4 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 2 个进程，每个进程将开启 3 个 openMP 线程，此作业将申请到 4*3=12 个 CPU 核心。

> 假如程序中指定了线程数，**OMP_NUM_THREADS** 环境变量将不会生效。
>
> 尽量使线程数接近每个进程申请的 CPU 核心数，即：**--cpus-per-task** 所指定的参数等于 **OMP_NUM_THREADS** 的值，如不指定 **--cpus-per-task** 参数的值，每个进程将只会使用一个 CPU 核心，不利于多线程作业的执行。

4.6.4 示例4 单节点GPU程序

```
#!/bin/bash
#SBATCH --job-name=test1                 # 指定作业名
#SBATCH -o result.out                    # 指定输出文件名;默认为slurm-[jobID].out
#SBATCH -e error.out                     # 指定程序错误输出文件名
#SBATCH --partition=q_intel_gpu_nvidia   # 指定作业提交的分区
#SBATCH --gres=gpu:1                     # 指定作业使用卡的总数为 1

# 加载cuda模块
module load amd/cuda/11.8.89

# 使用nvcc编译cuda脚本
nvcc -o test1 test1.cu

## Run the script
./test1
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_intel_gpu_nvidia 队列申请使用 1 张 gpu 卡运行程序，不指定节点数默认在单节点运行

4.7 查看作业

使用 `squeue` 查看当前集群的作业队列

使用 `myjob` 可以只查看自己的作业

4.7 取消作业

取消作业的命令为 `scancel jobid`，或使用 `bkill jobid`

如需取消账户的所有作业，使用 `scancel -u username`

4.8 查找历史作业

使用 `sacct` 命令可以查看历史作业情况

示例：

```
sacct -S 2022-10-15T00:00:00 -o
"jobid,jobname,partition,user,alloccpus,state,nodelist,start,end,workdir%100" -X
```

该命令将查询从 2022 年 10 月 15 日 0 点以后的所有作业的作业号、作业名，所提交队列、账户名、申请的 CPU 核心数、作业状态、所使用的节点主机号、作业开始时间、作业结束时间、作业的提交目录（最长显示 100 个字符）。

如需进行二次模糊查找，可以使用管道符和 grep 进行筛选过滤。

执行 `scontrol show job jobid` 可以查看对应 `jobid` 的作业详情，如工作目录，运行时间等等

> 提交作业
>
> 2. 开启独占将按节点核心总数进行计算核时费，而非按照程序实际使用的核心数目进行计费

该命令将向 q_amd_share 队列申请 bn[013-015,017,023-028] 这 10 个节点运行 mpi_hello 程序，且该作业独占这 10 个节点，-n 10 指定了运行该程序的总进程数，这 10 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 1 个进程，如图：

```
[qiql@psn001 online1]$ srun -p q_amd_share -N 10 -n 10 --nodelist=bn[013-015,017,023-028] --exclusive  ./mpi_hello
Hello world from processor bn027, rank 8 out of 10 processors
Hello world from processor bn025, rank 6 out of 10 processors
Hello world from processor bn026, rank 7 out of 10 processors
Hello world from processor bn023, rank 4 out of 10 processors
Hello world from processor bn028, rank 9 out of 10 processors
Hello world from processor bn024, rank 5 out of 10 processors
Hello world from processor bn013, rank 0 out of 10 processors
Hello world from processor bn014, rank 1 out of 10 processors
Hello world from processor bn017, rank 3 out of 10 processors
Hello world from processor bn015, rank 2 out of 10 processors
[qiql@psn001 online1]$
```

4.5.3 示例3 混合 MPI / openMP 程序

假如程序中的并行模式为 MPI + openMP，那么需要定义环境变量 **OMP_NUM_THREADS** 来控制每个进程所开启的线程数

```
export OMP_NUM_THREADS=3 # 每个进程开启 3 个线程

srun -p q_amd_share -N 2 -n 4 --nodelist=bn[013-014] --ntasks-per-node=2 --cpus-per-task=3  ./mpi_openmp_hello
```

> 环境变量 **OMP_NUM_THREADS** 用来指定一个进程的线程数目，此环境变量**不一定生效**，如果程序中已指定了线程数目，那么将**覆盖**此环境变量所设置的值
>
> tips: openMP 程序中不指定线程数目的方法：
> 代码中将指令 **# pragma omp parallel num_threads(threads_count)** 替换为：**# pragma omp parallel** 即可

该命令将向 q_amd_share 队列申请 bn013 和 bn014 这 2 个节点运行 mpi_openmp_hello 程序，其中：

-N 2 指定了运行该程序的总节点数；

-nodelist=bn[013-014] 指定了运行程序的节点号范围为 bn013 和 bn014

-n 4 指定了运行该程序的总进程数；

--ntasks-per-node=2 指定了每个节点的进程数；

--cpus-per-task=3 指定了每个进程使用的 CPU 核心数，若不指定该参数，默认为 1，即每个进程默认使用 1 个 CPU 核心；

在此示例中，每个节点将运行 2 个进程，每个进程运行 3 个线程，共使用 12 个CPU 核心，如图：

```
[qiql@psn001 guide]$ srun -p q_amd_share -N 2 -n 4 --nodelist=bn[023-024]   --ntasks-per-node=2 --cpus-per-task=6  ./mpi_openmp_hello
Hello world from thread 0 ,thread num: 3 ; processor bn023, rank 0 out of 4 processors
Hello world from thread 2 ,thread num: 3 ; processor bn023, rank 0 out of 4 processors
Hello world from thread 1 ,thread num: 3 ; processor bn023, rank 0 out of 4 processors
Hello world from thread 0 ,thread num: 3 ; processor bn023, rank 1 out of 4 processors
Hello world from thread 2 ,thread num: 3 ; processor bn023, rank 1 out of 4 processors
Hello world from thread 1 ,thread num: 3 ; processor bn023, rank 1 out of 4 processors
Hello world from thread 0 ,thread num: 3 ; processor bn024, rank 2 out of 4 processors
Hello world from thread 1 ,thread num: 3 ; processor bn024, rank 2 out of 4 processors
Hello world from thread 2 ,thread num: 3 ; processor bn024, rank 2 out of 4 processors
Hello world from thread 0 ,thread num: 3 ; processor bn024, rank 3 out of 4 processors
Hello world from thread 1 ,thread num: 3 ; processor bn024, rank 3 out of 4 processors
Hello world from thread 2 ,thread num: 3 ; processor bn024, rank 3 out of 4 processors
[qiql@psn001 guide]$
```

4.5.4 示例4 单节点 GPU 程序

目前每个 GPU 节点上有 8 张卡

假如作业使用的卡数量小于 8 ，即只使用1 个节点，那么srun提交命令为：

```
module load amd/cuda/11.8.89 # 加载 cuda11.8.89 环境

srun -p q_intel_gpu_nvidia --gres=gpu:1 ./hello_world
```

该命令将向 q_intel_gpu_nvidia 队列申请 1 张卡运行 hello_world 程序

```
[test001@psn001 online1]$ srun -p q_intel_gpu_nvidia  -n 1 --gpus-per-task=1 hello_world
Hello World from host!
Hello World! from thread [0,0]          From device
```

4.6 后台提交作业

如希望断开会话连接后作业仍在运行，建议采用 `sbatch` 进行后台提交作业

后台提交需要用户先编写作业脚本，指定一些参数来申请程序所使用的节点数，进程数，线程数等信息。脚本名称无限制（一般为 run.sh），编写完后，使用 **sbatch 脚本名** 提交作业。脚本第一行默认为 `#!/bin/bash` ，请勿随意更改。

以下为脚本示例，您可以根据您的需要进行参数的删除或修改

```
#!/bin/bash
#SBATCH --job-name=JOBNAME       #指定作业名称
#SBATCH -p q_amd_share           #指定分区
#SBATCH -N 2                     #指定节点数量
#SBATCH -n 32                    #指定总进程数
#SBATCH -G 8                     #指定程序使用GPU数【可选】
#SBATCH --gpus-per-task=1        #指定每个任务所使用的GPU数【可选】
#SBATCH -o job.out               #指定输出文件输出
#SBATCH -e job.error.out         #指定错误文件输出
#SBATCH --ntasks-per-node=16     #指定每个节点进程数【可选】
#SBATCH --cpus-per-task=1        #指定每个进程使用 CPU 核心数，不指定默认为 1 【可选】
#SBATCH --nodelist=node[3,4]     #指定优先使用节点【可选】
#SBATCH --exclude=node[1,5-6]    #指定避免使用节点【可选】
#SBATCH --exclusive              #独占该节点【可选】

unset I_MPI_PMI_LIBRARY          # 使用mpirun运行程序，需卸载此环境变量

module load 模块名               #加载软件或其他环境变量
mpirun -n 32 test                #作业运行命令
```

4.6.1 示例 1 非 MPI 程序

编写作业脚本 run.sh（脚本名名称无限制）

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH -p q_amd_share                      # 指定作业提交的分区
#SBATCH -N 1                                # 指定节点数量为 1
#SBATCH -n 1                                # 指定总进程数为 2
#SBACTH -o out                              # 指定作业输出文件名 %j 是作业号
#SBATCH -e %j_.error.out                    # 指定作业中的错误所输出的文件名，%j 是作业
号
#SBATCH --ntasks-per-node=1                 # 每个节点开启 1 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                   # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn114                    # 指定所使用的的节点【可选】
#SBATCH --exclusive                         # 使该作业独占该节点【可选】

hostname                                    # 作业运行命令
```

执行 `sbatch run.sh` 进行作业提交

该作业将向 q_amd_share 队列的 bn114 节点申请 1 个 CPU 核心，且独占该节点，程序使用 1 个进程，每个进程只开启 1 个线程

4.6.2 示例 2 单节点或跨节点 MPI 程序

编写作业脚本 run.sh（脚本名称无限制）

> **注意：使用 mpirun 运行程序时，需要在 mpirun 命令前执行 unset I_MPI_PMI_LIBRARY**

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH -p q_amd_share                      # 指定作业提交的分区
#SBATCH -N 1                                # 指定节点数量为 1
#SBATCH -n 2                                # 指定总进程数为 2
#SBACTH -o job.%j.out                       # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                    # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH --ntasks-per-node=2                 # 每个节点开启 2 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                   # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn114                    # 指定所使用的的节点【可选】
#SBATCH --exclusive                         # 使该作业独占该节点【可选】

unset I_MPI_PMI_LIBRARY                     # 使用 mpirun 运行程序，需卸载此环境变量
module load amd/intel_compiler/2019u5       # 加载环境
mpiicc -o mpi_hello ./mpi_hello.c           # 编译程序
mpirun -n 2 ./mpi_hello                     # 运行程序
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_amd_share 队列申请 1 个节点运行程序，指定节点为 bn114 且独占该节点

假如MPI程序需要多节点并行，修改作业脚本为：

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                # 指定作业名
#SBATCH --partition=q_amd_share             # 指定作业提交的分区
#SBATCH -N 10                               # 指定节点数量为 10
```

```
#SBATCH -n 20                                  # 指定总进程数为 20
#SBACTH -o job.%j.out                          # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                       # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH --ntasks-per-node=2                    # 每个节点开启 2 个进程，不指定时，slurm
会尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=1                      # 每个进程使用 1 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --nodelist=bn[013-015,017,023-028] # 指定所使用的的节点【可选】
#SBATCH --exclusive                            # 使该作业独占该节点【可选】

unset I_MPI_PMI_LIBRARY                        # 使用 mpirun 运行程序时，需卸载此环境变量
module load amd/intel_compiler/2019u5          # 加载环境
mpiicc -o mpi_hello ./mpi_hello.c              # 编译程序
mpirun -n 20 ./mpi_hello                       # 运行程序
```

该作业脚本将向 q_amd_share 队列申请 bn[013-015,017,023-028] 这 10 个节点运行 mpi_hello 程序，且该作业独占这 10 个节点，-n 10 指定了运行该程序的总进程数，这 10 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 1 个进程。

4.6.3 示例 3 混合 MPI / openMP 程序

假如程序需要用到 **openMP** 开启多线程并行，需要在作业脚本中使用 **OMP_NUM_THREADS** 环境变量来指定线程数目。

编写作业脚本 `run.sh`（脚本名称无限制）

> **注意：使用 mpirun 运行程序时，需要在 mpirun 命令前执行** `unset I_MPI_PMI_LIBRARY`

```
#!/bin/bash
#SBATCH --job-name=mpi_hello                   # 指定作业名
#SBATCH --partition=q_amd_share                # 指定作业提交的分区
#SBATCH -N 2                                   # 使用 2 个节点
#SBATCH --nodelist=bn[013-014]                 # 指定所使用的的节点
#SBATCH -o %j.out                              # 指定作业输出文件名 %j是作业号
#SBATCH -e %j_.error.out                       # 指定作业中的错误所输出的文件名，%j是作业号
#SBATCH -n 4                                   # 指定进程总数为 4
#SBATCH --ntasks-per-node=2                    # 每个节点使用 2 个进程，不指定时，slurm 会
尽量均匀的分配进程数到所申请的节点上【可选】
#SBATCH --cpus-per-task=3                      # 每个进程使用 3 个 CPU 核心，不指定默认为
1 【可选】
#SBATCH --exclusive                            # 使该作业独占该节点【可选】

export OMP_NUM_THREADS=3                       # 每个进程开启 3 个线程（不一定生效，如果程序
中指定了线程数量，将覆盖此环境变量）
unset I_MPI_PMI_LIBRARY                        # 使用 mpirun 运行程序时，需先卸载此环境变量
module load amd/intel_compiler/2019u5          # 加载环境
mpiicc -qopenmp -o mpi_openmp_hello ./mpi_openmp_hello.c  # 编译程序命令
mpirun -n 4 ./mpi_openmp_hello             # 程序运行命令
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_amd_share 队列申请 bn[013-014] 这 2 个节点运行 mpi_openmp_hello 程序，且该作业独占这 2 个节点，该作业运行完毕之前，其他用户无法向这 2 个节点提交作业，-n 4 指定了运行该程序的进程总数，这 4 个进程将尽量均匀的分别在各个节点上，在此示例中，每个节点将运行 2 个进程，每个进程将开启 3 个 openMP 线程，此作业将申请到 4*3=12 个 CPU 核心。

> 假如程序中指定了线程数，**OMP_NUM_THREADS** 环境变量将不会生效。

> 尽量使线程数接近每个进程申请的 CPU 核心数，即：**--cpus-per-task** 所指定的参数等于 **OMP_NUM_THREADS** 的值，如不指定 **--cpus-per-task** 参数的值，每个进程将只会使用一个 CPU 核心，不利于多线程作业的执行。

4.6.4 示例4 单节点GPU程序

```
#!/bin/bash
#SBATCH --job-name=test1                # 指定作业名
#SBATCH -o result.out                   # 指定输出文件名;默认为slurm-[jobID].out
#SBATCH -e error.out                    # 指定程序错误输出文件名
#SBATCH --partition=q_intel_gpu_nvidia  # 指定作业提交的分区
#SBATCH --gres=gpu:1                    # 指定作业使用卡的总数为 1

# 加载cuda模块
module load amd/cuda/11.8.89

# 使用nvcc编译cuda脚本
nvcc -o test1 test1.cu

## Run the script
./test1
```

执行 `sbatch run.sh` 进行作业提交

该作业脚本将向 q_intel_gpu_nvidia 队列申请使用 1 张 gpu 卡运行程序，不指定节点数默认在单节点运行

4.7 查看作业

使用 `squeue` 查看当前集群的作业队列

使用 `myjob` 可以只查看自己的作业

4.7 取消作业

取消作业的命令为 `scancel jobid`，或使用 `bkill jobid`

如需取消账户的所有作业，使用 `scancel -u username`

4.8 查找历史作业

使用 `sacct` 命令可以查看历史作业情况

示例：

```
sacct -S 2022-10-15T00:00:00 -o
"jobid,jobname,partition,user,alloccpus,state,nodelist,start,end,workdir%100" -X
```

该命令将查询从 2022 年 10 月 15 日 0 点以后的所有作业的作业号、作业名，所提交队列、账户名、申请的 CPU 核心数、作业状态、所使用的节点主机号、作业开始时间、作业结束时间、作业的提交目录（最长显示 100 个字符）。

如需进行二次模糊查找，可以使用管道符和 grep 进行筛选过滤。

执行 `scontrol show job jobid` 可以查看对应 `jobid` 的作业详情，如工作目录，运行时间等等